

"2차 과제 코드를 함수 기반으로 바꾸고 싶어. while True 메뉴 구조에 맞게 입력/조회/분석 기능을 각각 def로 분리해줘. 매개변수와 return도 써야 해."

좋아요! 2차 과제 코드를 함수 기반으로 구조화해볼게요.

핵심 설계 방향은 이렇게 잡았어요:

`while True` 무한루프를 메인 뼈대로 삼고, 그 안에서 함수 호출만 하도록 구성하는 거예요. 마치 목차처럼요.

함수 분리 구조:

`input_trade()` — 매매 기록 입력. `global` 키워드로 전역변수 `trade_history`와 `total_sum`을 수정해요.

`show_trades()` — 전체 기록 조회. `enumerate`로 번호 붙여서 표 형태로 출력해요.

`calc_profit_rate(buy, sell)` — 매개변수로 매수가/매도가를 받아서 수익률을 계산하고 `return`으로 반환해요.

`get_grade(profit_rate, invest_amount)` — `and` 논리 연산자로 복합 조건을 걸어서 등급과 칭호를 튜플로 반환해요.

`show_analysis()` — 위 두 함수를 호출해서 전체 분석 리포트를 출력해요.

메인 루프는 이렇게 간결해져요:

```
while True → 메뉴 출력 → 선택 입력
→ 1: input_trade() 호출
→ 2: show_trades() 호출
→ 3: show_analysis() 호출
→ 4: break
```

"함수 안에서 trade_history 리스트를 수정하려고 하는데 왜 밖에서 변경이 안 돼?
global은 언제 써야 해?"

좋은 질문이에요!

파이썬에서 함수는 기본적으로 전역변수를 읽을 수는 있지만, 재할당(수정)은 못해요.

예를 들면 이런 상황이에요:

```
python

total_sum = 0.0 # 전역변수

def input_trade():
    total_sum += 100 # ❌ 에러! 함수는 자기만의 지역변수로 인식해버림
```

파이썬이 `total_sum`을 함수 안의 지역변수라고 착각해서 `UnboundLocalError`가 나요.

`global`을 쓰면 해결돼요:

```
python

def input_trade():
    global trade_history, total_sum # ✅ "이건 전역변수야!" 라고 선언
    total_sum += 100 # 이제 전역변수 수정 가능
```

한 줄 요약으로는:

- 전역변수 읽기만 → `global` 없어도 됨
- 전역변수 수정(+=,=) → 반드시 `global` 선언 필요

그래서 `input_trade()` 함수 맨 위에 `global trade_history, total_sum`을 선언한 거예요!

📄 👍 💬 ↺



매수가가 0일 때 문제가 생기는데 문제가 생기는데 해결 하는 법 알려줘

오전 1:57 ↺ ✎ 📄

`calc_profit_rate` 함수에서 `buy` 가 0이면 `ZeroDivisionError`가 나요!

python

```
profit_rate = ((sell - buy) / buy) * 100
```

```
#                                     ↑ 여기서 0으로 나누기 → 프로그램 강제 종료
```

해결법은 함수 첫 줄에 조건 추가하면 돼요:

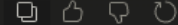
python

```
def calc_profit_rate(buy, sell):  
    if buy == 0:           # 매수가가 0이면  
        return 0.0        # 계산 안 하고 바로 0 반환  
    profit_rate = ((sell - buy) / buy) * 100  
    return profit_rate
```

`if buy == 0` 체크를 먼저 해서 0일 때는 계산 자체를 건너뛰는 거예요. 이렇게 하면 프로그램이 멈추지 않고 안전하게 실행돼요!

📄 🍎 💬 ↺





함수 밖에서 선언한 리스트가 함수 안에서 수정이 안돼

아까 `global` 질문이랑 같은 문제예요!

`trade_history` 는 리스트라서 조금 헷갈릴 수 있는데, 정확히 설명하면:

리스트 내용 변경 (`append`, `insert`, `pop`) → `global` 없어도 됨

```
python
def input_trade():
    trade_history.append([...]) # ✅ 이걸 괜찮음
```

리스트 자체를 재할당 → `global` 필요

```
python
def input_trade():
    trade_history = [] # ❌ 이러면 지역변수로 인식
```

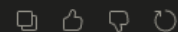
근데 `total_sum` 처럼 `+=` 쓰는 건 반드시 `global` 필요:

```
python
def input_trade():
    global trade_history, total_sum
    total_sum += buy * amount # ✅ global 없으면 에러
```

그래서 `input_trade()` 함수 맨 위에 이렇게 선언한 거예요:

```
python
global trade_history, total_sum
```

둘 다 묶어서 한 줄로 선언하면 깔끔하게 해결돼요! 😊





각 박스를 클릭하면 해당 함수에 대한 질문을 바로 할 수 있어요!

전체 코드 구조 한눈에:

- 맨 위: 전역변수 (`trade_history`, `total_sum`) — 전체에서 공유
- 중간: 5개 함수 — teal(입출력용), purple(계산+반환용)으로 색 구분
- 아래: `while True` 메인 루프 → 메뉴 선택 → 해당 함수 호출

```

if choice == '1':
    input_trade()      # 함수 호출
elif choice == '2':
    show_trades()      # 함수 호출
elif choice == '3':
    show_analysis()    # 함수 호출
elif choice == '4':
    print('\n 🍀 프로그램을 종료합니다. 성공적인 투자 되세요!')
    break              # 종료 메뉴 선택 시 break
else:
    print(' ⚠️ 1~4 중에서 선택해 주세요.')

```

지금 내 코드에서 뭐가 문제인거지?

간략히 보기

버그 2개 찾았어요! 빠르게 정리할게요.

버그 1: 함수 이름 오타

```

python

# 함수 정의할 때
def show_traders():  # ← s 붙어있음

# 근데 호출할 때
show_trades()        # ← s 없음 → 에러남!

```

`show_traders` → `show_trades` 로 둘 중 하나 통일하면 돼요.

버그 2: `\n` 위치 오타

```

python

print('/n--- 📊 전체 매매 기록 조회 ---')
#      ↑ 슬래시(/) → 줄바꿈 안 됨, /n 이 그대로 출력됨

# 이렇게 고쳐야 해요
print('\n--- 📊 전체 매매 기록 조회 ---')
#      ↑ 백슬래시(\)

```

이 두 개만 고치면 정상 실행돼요! 나머지 로직은 다 맞게 잘 짜셨어요 🍀